Declaration of Originality and Submission Information

I affirm that this submission is my own / the groups original work in accordance with the University of Glasgow Regulations and the School of Engineering Requirements.

1. **Student Number**: 2544946D **Student Name**: Prateek Singh Dadial

2. **Student Number**: 2509905A **Student Name**: Shawn Anto Thekkiniath

Assignment 1

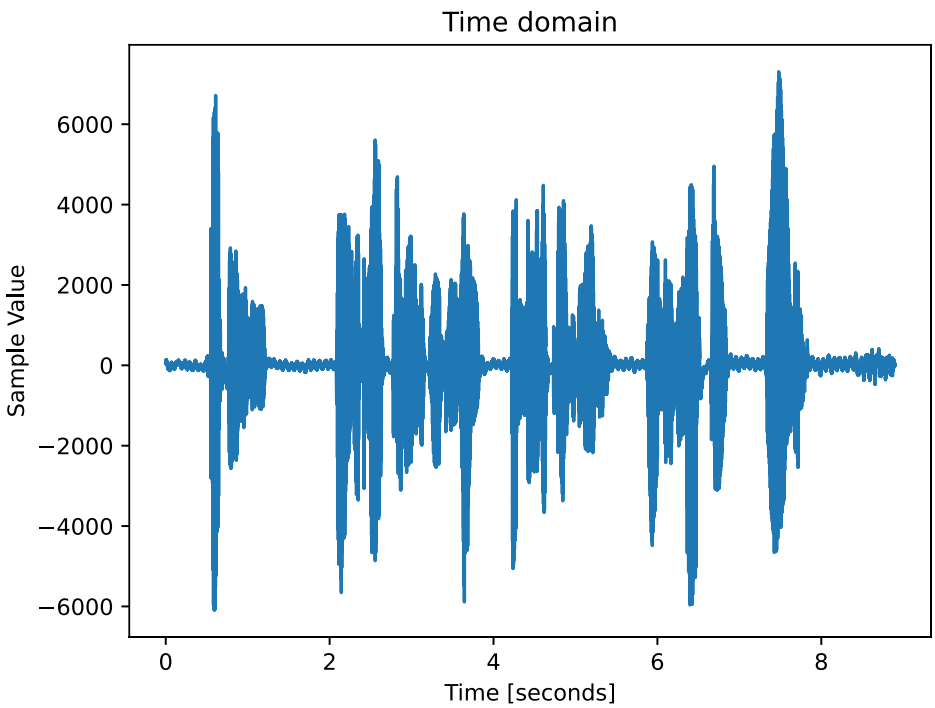
Question 2:

The recorded WAV file "*original.wav*" is stereo, sampled at 44100 Hz and is 8.8 seconds long. Convert to mono channel.

```
fs, data = wavfile.read("original.wav")
data = data[:,0] #conversion to mono channel
```

Plot in time domain with the x-axis of the plot defined as:

```
t = np.linspace(0, len(data)/fs, len(data))
```

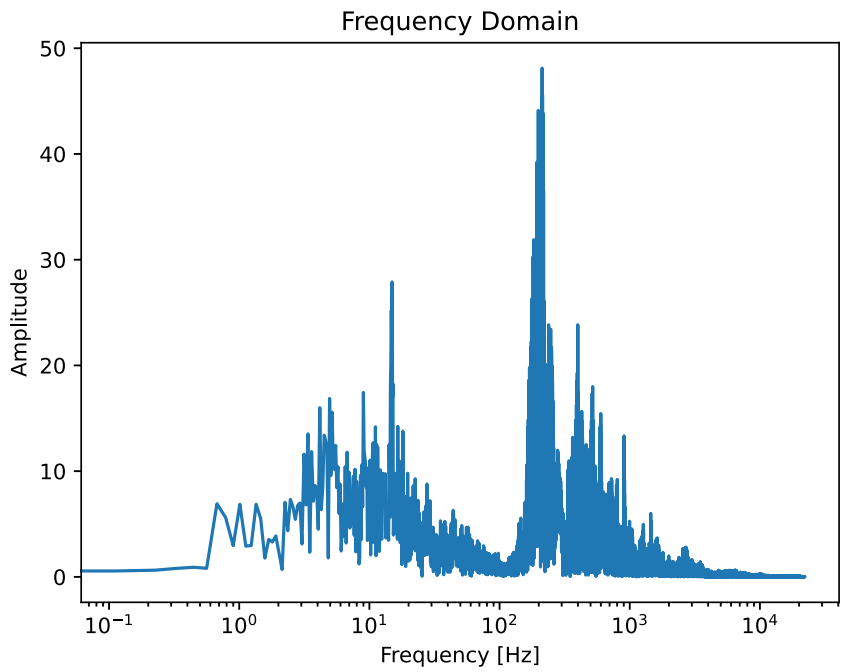


The x-axis represents time in seconds, and the y-axis represents sample values.

Transform to frequency domain using Fast Fourier Transformation. Modify the output before plotting:

- Consider the magnitude of the complex frequency function, as it combines the real and imaginary components and tells the amount of energy in each frequency bin.
- Cut the frequency spectrum at Nyquist, as anything above that is a mirror.
- Divide amplitude by the total number of samples (N) to normalise and compensate for the coefficient accumulation during calculation of DFT.

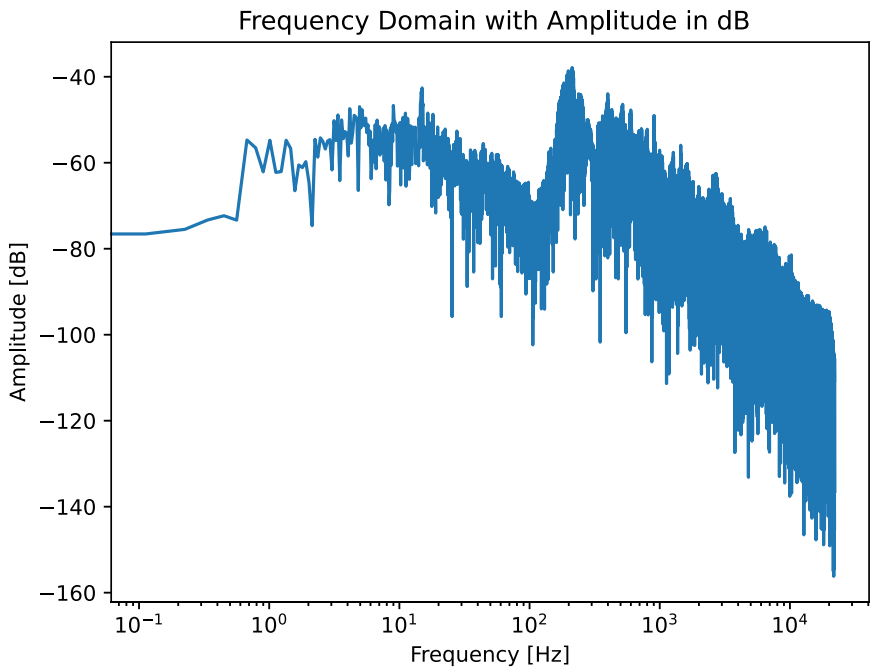
```
X = abs(np.fft.fft(data))
N = len(X)
X_mirror = X[0:int(N/2)] / N
```



The x-axis represents frequency in Hz (log scale), and the y-axis represents amplitude.

Convert amplitude to dB, with 0dB as 32678 (int16 signal and plot in frequency domain.

```
ref = 3762 # 0 dB is 32678 with an int16 signal
X_dB =  20 * np.log10(X_mirror / ref) # convert to dB
```



The x-axis represents frequency in Hz (log scale), and the y-axis represents amplitude (in dB).

Question 3:

The fundamental frequency is the frequency at which the vocal cords are opening and closing. Any other major frequency components present in the speech are integer multiples of fundamental frequency and are called harmonics. There are individual differences in fundamental frequencies of adult men and women. These differences range from 80-175 Hz among men and 160-270 Hz among women. Therefore, it can be assumed that the fundamental frequency of spoken words lies in the range of 80-300Hz (approximately).

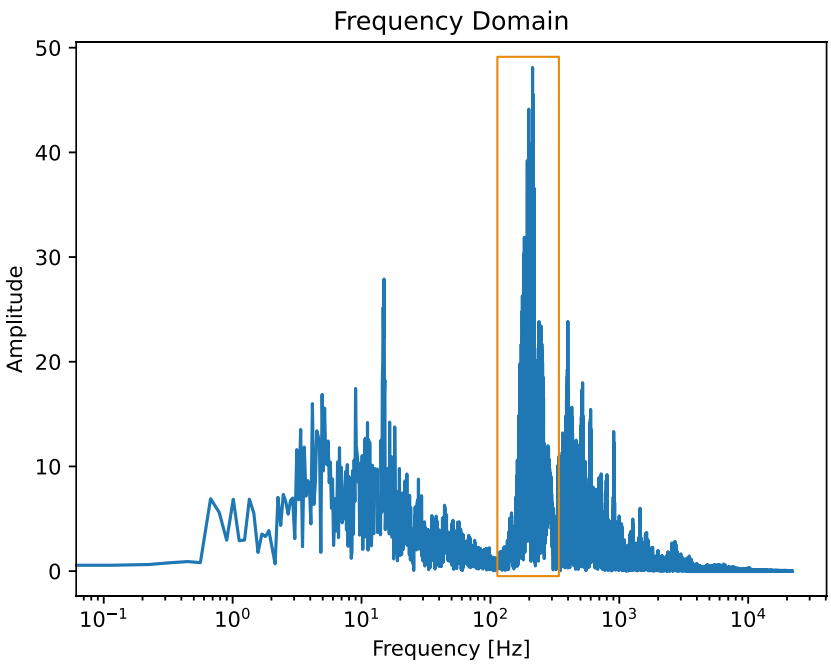
The frequencies below this bandwidth can be considered as low frequency noise. Their frequency is so low that they become inaudible to human ear. Low frequency noise typically ranges between 10-100 Hz and becomes difficult to control. Thus, they can be neglected.

The range of frequencies for the vowels and consonants present in the speech can be identified from the frequency domain. A speech can be classified into two categories: Voiced and Unvoiced speech signals.

Voiced signals are produced when the vocal cords vibrate during the pronunciation of a phenome. In contrast, unvoiced signals do not entrail the use of vocal cords.

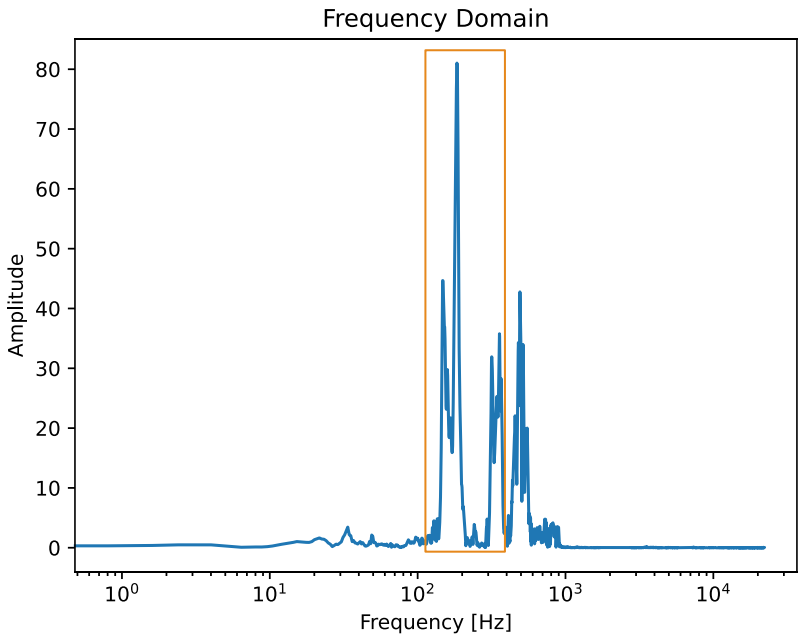
- Identification of Vowels:

Vowels are voiced signals. They are periodic signals with higher energy (louder) and tend to lie within the fundamental frequency range of the voice.



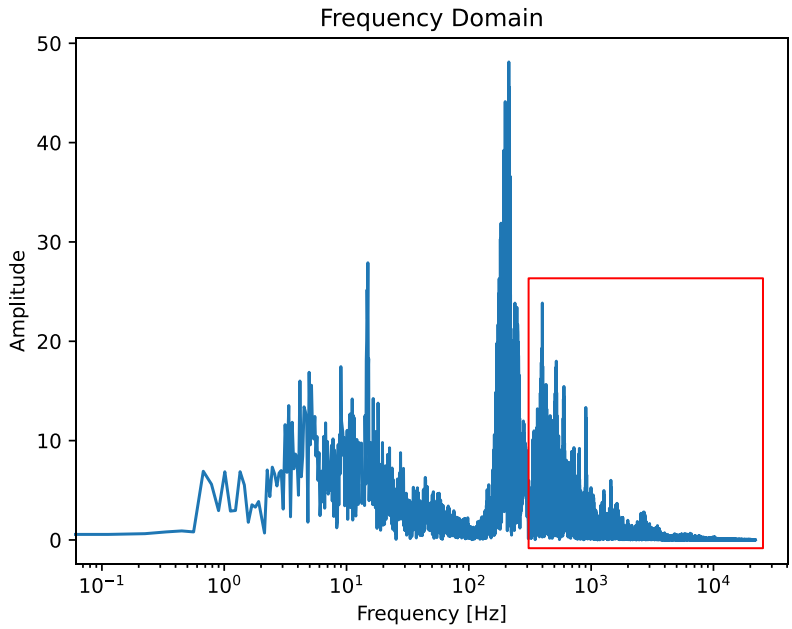
In the above figure, the frequencies enclosed within the orange box represents the frequency range of vowels spoken. The peaks represent the fundamental frequency of the vowels. This can be further verified by comparing to a single vowel in the frequency domain.

The frequency domain of vowel *'a'* is given below:



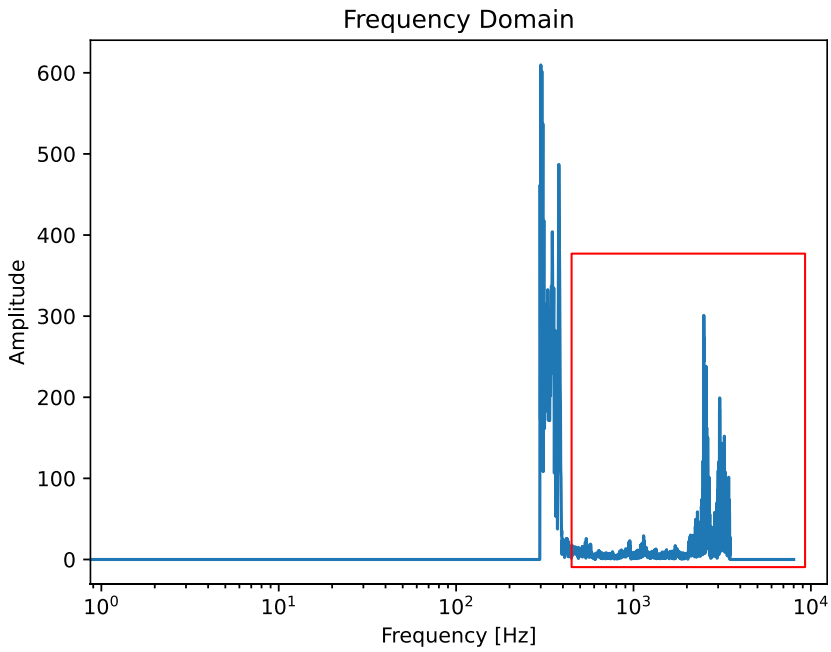
- Identification of consonants:

Consonants are unvoiced signals. They are plosive in nature and tend to have lower energy and higher frequency range in contrast to vowels.



In the above figure, the range of frequencies within the red box represents the consonants present in the speech signal. This can be further verified by comparing to a single consonant in the frequency domain.

The frequency domain of vowel *'b'* is given below:



Question 4:

Using Fast Fourier Transform the amplitudes of the harmonics are increased to improve the quality of the recorded WAV file.

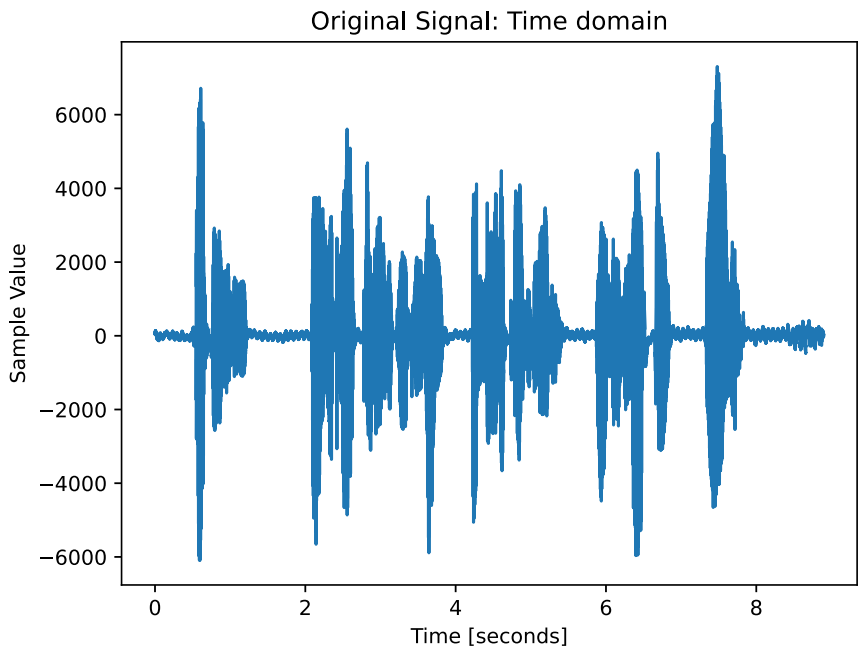
Frequencies between 90 Hz and 150 Hz are taken as fundamental frequencies. Harmonics lying between 6000 Hz and 10000 Hz are boosted and frequencies up to 90 Hz, considered as low frequency noise, are attenuated by using suitable gain factor.

The value of gain factors was decided by trial and error method, and those with the best results were considered.

Plot the original signal in time domain with the x-axis as time in seconds.

```
fs,data = wavfile.read("original.wav")
data = data[:,0] # convert to mono channel
t = np.linspace(0,len(data)/fs,len(data)) # time axis
```

```
plt.plot(t, data)
```

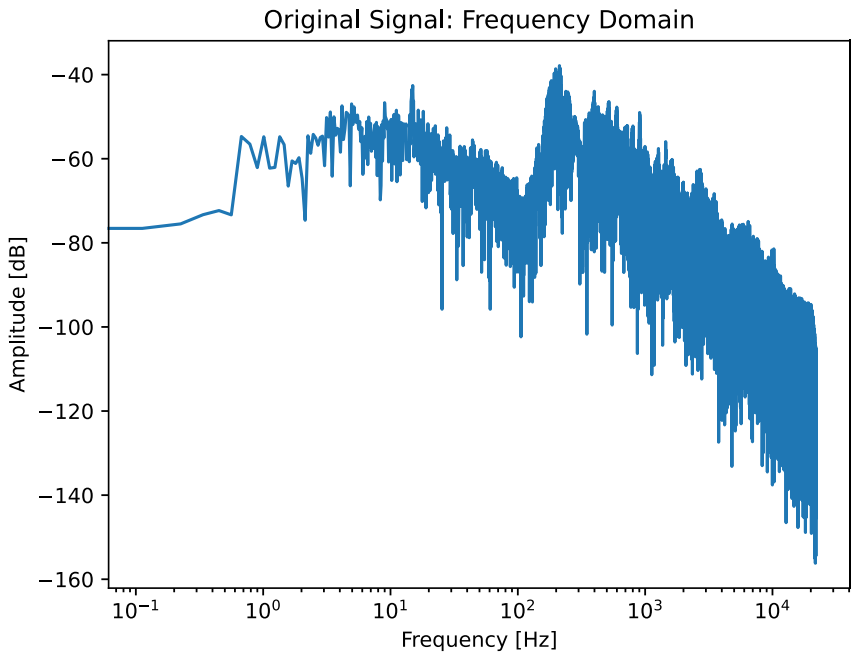


Transform to frequency domain using FFT and plot the frequency spectrum of original signal.

```
X = np.fft.fft(data)
N = len(X)
freq = np.linspace(0, fs/2, int(N/2)) # frequency axis
```

```
ref = 3762 # 0 dB is 32678 with an int16 signal
```

```
plt.plot(freq, 20 * np.log10((abs(X)/len(data)) / ref)[0:int(N/2)])
```



Calculate frequency resolution by dividing sampling frequency with total number of samples in FFT. Frequency resolution is the difference in frequency between each bin and is used to find the index value of frequencies.

```
res = fs/N # frequency resolution
```

Increase the amplitude of the harmonics in range 6-10 kHz by a factor of 10.

```
f1 = int(np.round(6000/res)) # index place for 6000Hz
f2 = int(np.round(10000/res)) # index place for 10000Hz
X[f1:f2] = X[f1:f2] * 10
```

The FFT yields a complex vector of coefficients. Thus, to respect the inherent Hermitian symmetry of the output of FFT, the amplitude is increased by conjugate pairs (except for *X[0]* which is DC offset).

```
X[(N-f2)+1:(N-f1)+1] = X[(N-f2)+1:(N-f1)+1] * 10
```

Increase the amplitude of the fundamental frequencies in range 90-150 Hz by 1.8.

```
f3 = int(np.round(90/res)) # index place for 90Hz
f4 = int(np.round(150/res)) # index place for 150Hz
X[f3:f4] = 1.8 * X[f3:f4]
```

```
X[(N-f4)+1:(N-f3)+1] = 1.8 * X[(N-f4)+1:(N-f3)+1] # increase amplitude by
conjugate pairs
```

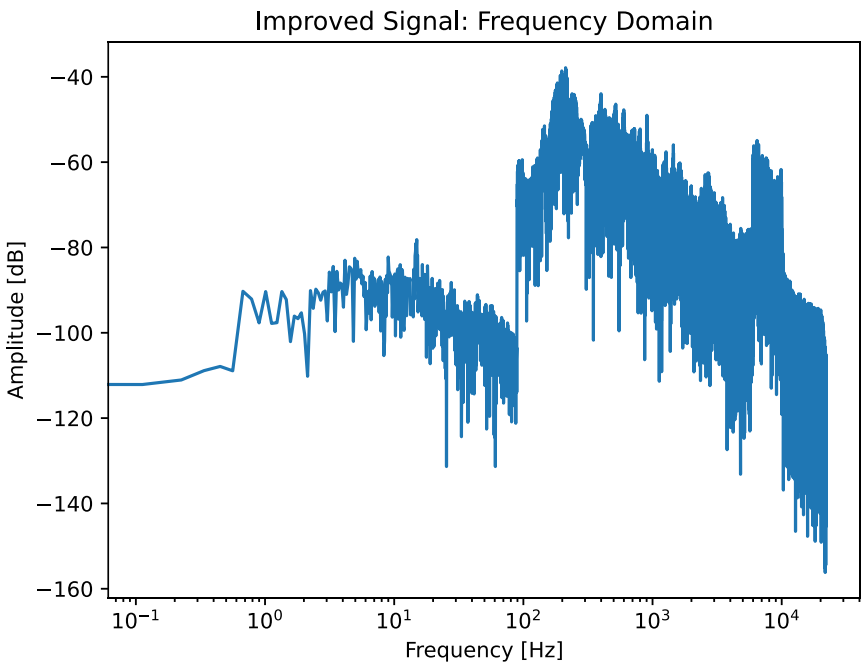
Decrease the amplitude of the low frequency noise in up to 90 Hz by 60.

```
n1 = int(np.round(0.1/res)) # index place for 0.1Hz
n2 = int(np.round(89/res)) # index place for 90Hz
X[n1:n2] = X[n1:n2] / 60
```

```
X[(N-n2)+1:(N-n1)+1] = X[(N-n2)+1:(N-n1)+1] / 60 # increase amplitude by
conjugate pairs
```

Plot the enhanced signal in frequency domain.

```
plt.plot(freq, 20 * np.log10((abs(X)/len(data)  / ref))[0:int(N/2)])
```

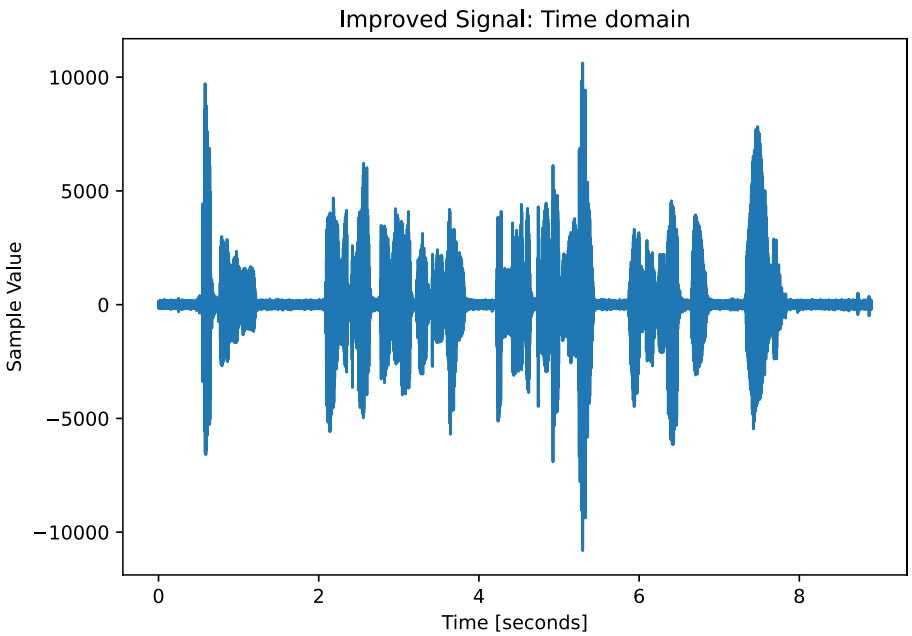


Transform the signal from frequency domain back to the time domain using Inverse Fast Fourier Transform. The IFFT function returns a complex valued signal. The imaginary values of the samples are quite low (e-13) because of proper manipulation of conjugate pairs in the frequency spectrum. Thus, it can be safely converted to a real valued signal by neglecting all imaginary values. The signal is further converted into 16-bit integer values.

```
imp_data = np.real(np.fft.ifft(X)).astype('int16')
```

Plot the enhanced signal in time domain.

```
plt.plot(t, imp_data)
```



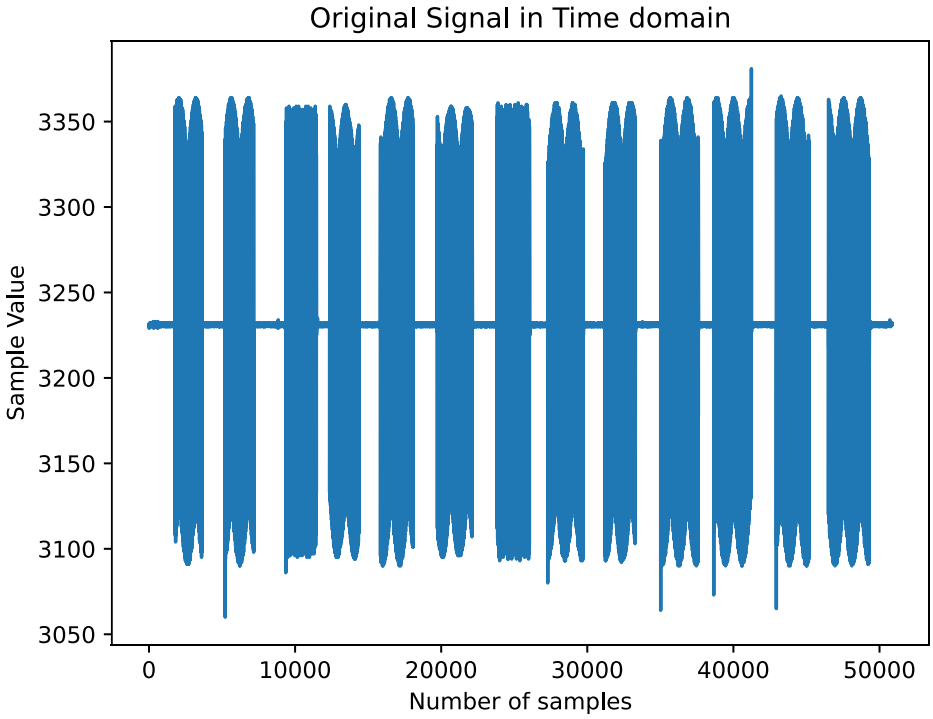
Write the enhanced audio file as *"improved.wav"* (16-bit).

```
wavfile.write("improved.wav",fs,imp_data)
```

Question 5:

Load the DTMF file into the variable *data* and set the sampling frequency as 1000 Hz.

```
data = np.loadtxt('touchtones.dat')
data = data[:,1]
Fs = 1000
```



Create a dictionary mapping each digit to a 4x3 matrix.

```
KEYS = [['1', '2', '3'], ['4', '5', '6'], ['7', '8', '9'], ['*', '0', '#']]
```

Normalise the signal between [1, -1] using equation:

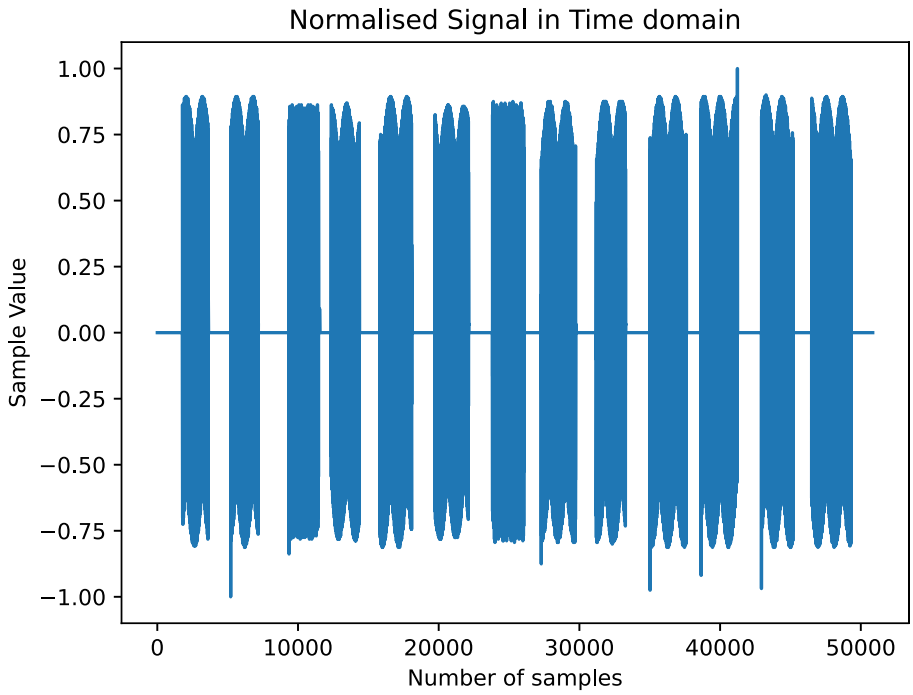
$$x_{norm} = \left((b-a) \times \left(\frac{x - x_{min}}{x_{max} - x_{min}}\right)\right) - a$$

Where '*a*' and '*b*' are *+1* and *-1* respectively.

```
norm_data = (2*((data-np.min(data))/(np.max(data)-np.min(data))))-1
```

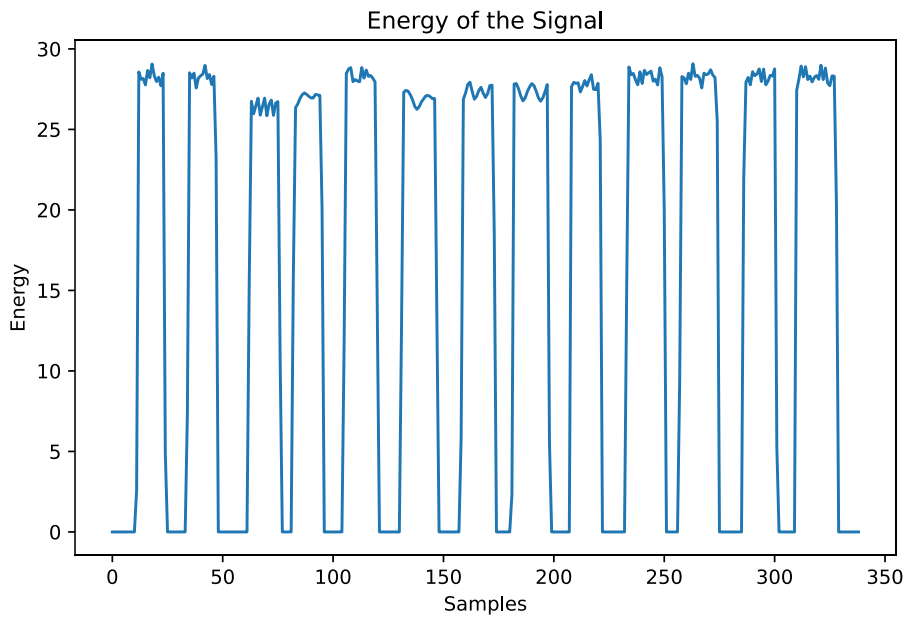
Remove noise between different keypresses for. Any sample value within range 0.04 to 0.085 is made equal to zero.

```
for i in range(len(norm_data)):
    if (norm_data[i] >= 0.04) and (norm_data[i] <= 0.085):
        norm_data[i] = 0
```



The function '*find_edges()*' takes the normalised signal as its argument and returns start and stop indices for each tone. It takes 150 samples chunk from the normalised data and arrange them as a matrix. The matrix is squared and added along the column to compute energy of each tone.

```
def find_edges(x, win = 150):
    w = np.reshape(x, (-1,win))
    # compute energy
    w_energy = np.sum(w*w,1)
```



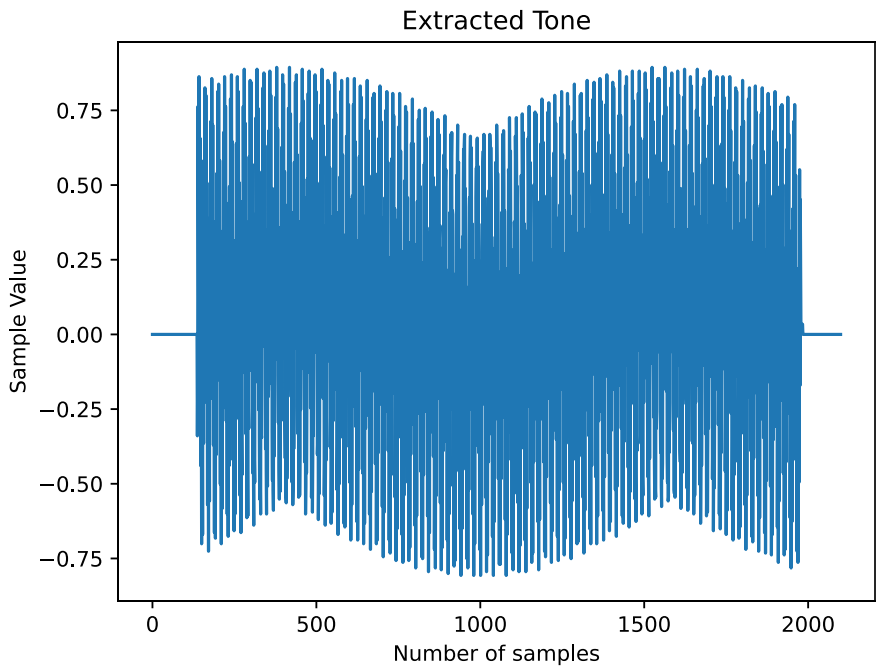
Create an empty list '*edges*' to contain start and stop indices of each tone. Loop through the energy signal '*w_energy*', finding every rising and falling edge. Append the list with respective index value while compensating for the window factor.

```
    edges=[]
    i=0
    j=0

    # loop for finding rising and falling edges in energy
    while (i < (len(w_energy))):
        while (i < len(w_energy) and w_energy[i] == 0):
            i = i + 1
        if (i >= len(w_energy)):
            break
        j = i
        while (j < len(w_energy) and w_energy[j] > 0):
            j = j + 1
        edges.append((i*win, j*win))
        i = j

    return edges
```

```
print(edges)
```

```
[(1650, 3750), (5100, 7200), (9300, 11550), (12300, 14400), (15750, 18150),
(19650, 22200), (23700, 26100), (27150, 29850), (31200, 33300), (34950,
37650), (38550, 41250), (42900, 45300), (46500, 49350)]
```

```
a,b = edges[0]
plt.plot(norm_data[a:b])
```



5a: Function that detects a digit from each tone segment.

The function '*decode_digit()*' takes the FFT of a tone, sampling frequency, number of samples and a dictionary mapping each digit to a 4x3 matrix, as its arguments, and returns the detected digit. Sort the indices of the peaks of the FFT signal in descending order. Extract the first two maximum peaks denoting the two frequencies in a tone.

```
def decode_digit(x, Fs, N, KEYS):
    Freqs = np.array([[697.0, 770.0, 852.0, 941.0],[1209.0, 1336.0, 1477.0,
0]])

    # resolution of each frequency bin
    res = float(Fs)/N

    # indices of peak values in dft
    peak_idx = np.argsort(-x)[0:2]
```

Create an empty list '*idx*' to contain rows and columns of each digit, mapped to the dictionary '*KEYS*'.

Loop through the array '*Freqs*'. Calculate distance between the frequency corresponding to the peak of FFT and the standard DTMF frequencies. The minimum distance gives the nearest standard DTMF frequencies in a tone. Append the list *'idx'* with the respective row and column value.

```
    idx = []
    # loop for finding the nearest DTMF frequencies in a tone
    for i in range(len(Freqs)):
        dist1 = abs(abs(Fs - Freqs[i]) - (peak_idx[0] * res))
        a =  np.argsort(dist1)[0]
        dist2 = abs(abs(Fs - Freqs[i]) - (peak_idx[1] * res))
        b =  np.argsort(dist2)[0]
        if dist1[a] < dist2[b]:
            idx.append(a)
        else:
            idx.append(b)

    digit = KEYS[idx[0]][idx[1]]
    return digit
```

The DTMF frequencies are higher than the sampling frequency of 1000 Hz, therefore, those frequencies are folded down to range 0 to 500 Hz (Nyquist frequency) using equation:

$$F_a = |F_s - F|$$

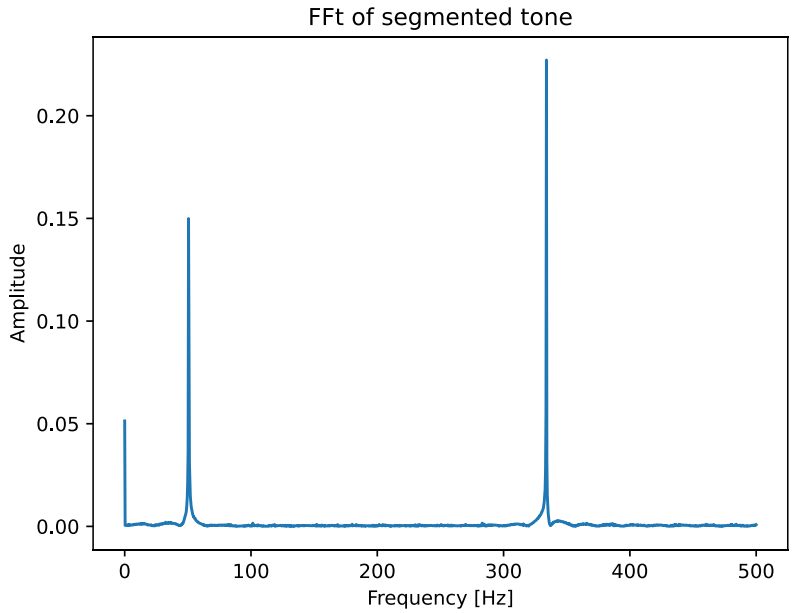
Where F_a is Aliasing frequency, F_s is Sampling frequency and F is DTMF frequency

5b: Main program as a loop which detects the key presses and then feeds them into the python function in 5a.

Main program to find start and stop indices of each individual tone using function *'find_edges()'*. The function returns a list *'edges'* containing the index values.

Loop through the list *'edges'*. Split signal *'norm_data'* into individual tone segments, calculate the FFT and feed to the function *'detect_digits()'*. The function returns a list containing the detected digits.

```
# main program

edges = find_edges(norm_data) # find edges of each individual tone
digits = []

# loop for splitting data into individual tone segments and detecting
digits
for edge in edges:
    toneFT = np.abs(np.fft.fft(norm_data[edge[0]:edge[1]])) # compute FFT
of tone segment
    N = len(toneFT)
    toneFT_mirror = toneFT[0:int(N/2)] / N
    digits.append(decode_digit(toneFT_mirror, Fs, N, KEYS))
```




```
print(digits)
```

```
['0', '0', '1', '2', '0', '2', '3', '5', '8', '0', '0', '0', '0']
```

Appendix

Question 2:

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.io.wavfile as wavfile

fs,data = wavfile.read("original.wav") # load the file
data = data[:,0] # select mono channel

# time domain representation
t = np.linspace(0, len(data)/fs, len(data)) # time axis
plt.figure(1)
plt.plot(t, data)
plt.xlabel('Time [seconds]')
plt.ylabel('Sample Value')
plt.title('Time domain')

# frequency domain representation
X = abs(np.fft.fft(data)) # transform to frequency domain
N = len(X)
X_mirror = X[0:int(N/2)] / N # half the FFT spectrum
freq = np.linspace(0, fs/2, int(N/2)) # frequency axis
plt.figure(2)
plt.plot(freq, X_mirror)
plt.xlabel('Frequency [Hz]')
plt.ylabel('Amplitude')
plt.title('Frequency Domain')
plt.xscale('log')

# frequency domain representation with amplitude in dB
ref = 3762 # 0 dB is 32678 with an int16 signal
X_dB = 20 * np.log10(X_mirror / ref) # convert to dB
plt.figure(3)
plt.plot(freq, X_dB)
plt.xlabel('Frequency [Hz]')
plt.ylabel('Amplitude [dB]')
plt.title(' Frequency Domain with Amplitude in dB')
plt.xscale('log')
plt.show()
```

Question 4:

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.io.wavfile as wavfile

fs,data = wavfile.read("original.wav")
data = data[:,0] # convert to mono channel
t = np.linspace(0,len(data)/fs,len(data)) # time axis

plt.figure(1)
plt.plot(t, data)
plt.xlabel('Time [seconds]')
plt.ylabel('Sample Value')
plt.title('Original Signal: Time domain')
```

```
X = np.fft.fft(data) # transform to frequency domain
N = len(X)
freq = np.linspace(0, fs/2, int(N/2)) # frequency axis
ref = 3762 # 0 dB is 32678 with an int16 signal

plt.figure(2)
plt.xlabel('Frequency [Hz]')
plt.ylabel('Amplitude [dB]')
plt.title('Original Signal: Frequency Domain')
plt.xscale('log')
plt.plot(freq, 20 * np.log10((abs(X)/len(data)) / ref)[0:int(N/2)])

res = fs/N # frequency resolution

# increase amplitude of frequencies around 6-10kHz
# by a factor of 10
f1 = int(np.round(6000/res)) # index place for 6000Hz
f2 = int(np.round(10000/res)) # index place for 10000Hz
X[f1:f2] = X[f1:f2] * 10
X[(N-f2)+1:(N-f1)+1] = X[(N-f2)+1:(N-f1)+1] * 10 # increase amplitude by
conjugate pairs

# increase amplitude of fundamental frequencies
# by a factor of 1.8
f3 = int(np.round(90/res)) # index place for 90Hz
f4 = int(np.round(150/res)) # index place for 150Hz
X[f3:f4] = 1.8 * X[f3:f4]
X[(N-f4)+1:(N-f3)+1] = 1.8 * X[(N-f4)+1:(N-f3)+1] # increase amplitude by
conjugate pairs

# reduce low frequency noise
# by a factor of 60
n1 = int(np.round(0.1/res)) # index place for 0.1Hz
n2 = int(np.round(89/res)) # index place for 90Hz
X[n1:n2] = X[n1:n2] / 60
X[(N-n2)+1:(N-n1)+1] = X[(N-n2)+1:(N-n1)+1] / 60 # increase amplitude by
conjugate pairs

plt.figure(3)
plt.xlabel('Frequency [Hz]')
plt.ylabel('Amplitude [dB]')
plt.title('Improved Signal: Frequency Domain')
plt.xscale('log')
plt.plot(freq, 20 * np.log10((abs(X)/len(data)  / ref))[0:int(N/2)])

imp_data = np.real(np.fft.ifft(X)).astype('int16') # transform back to time
domain
plt.figure(4)
plt.plot(t, imp_data)
plt.xlabel('Time [seconds]')
plt.ylabel('Sample Value')
plt.title('Improved Signal: Time domain')
plt.show()
wavfile.write("improved.wav",fs,imp_data)
```

Question 5:

```
import numpy as np
import matplotlib.pyplot as plt

# function that returns start and stop indices for each tone
def find_edges(x, win = 150):
    """

    Parameters
    ----------
    x : normalised data.
    win : window size, default is 150.

    Returns
    -------
    edges : start and stop indices.

    """
    w = np.reshape(x, (-1,win))
    w_energy = np.sum(w*w,1) # compute energy

    plt.figure(3)
    plt.plot(w_energy)
    plt.xlabel('Samples')
    plt.ylabel('Energy')
    plt.title('Energy of the Signal')

    edges=[]
    i=0
    j=0

    # loop for finding rising and falling edges in energy
    while (i < (len(w_energy))):
        while (i < len(w_energy) and w_energy[i] == 0):
            i = i + 1
        if (i >= len(w_energy)):
            break
        j = i
        while (j < len(w_energy) and w_energy[j] > 0):
            j = j + 1
        edges.append((i*win, j*win))
        i = j

    return edges

#function that detects a digit from each tone segment
def decode_digit(x, Fs, N, KEYS):
    '''

    Parameters
    ----------
    x : FFT of a tone.
    Fs : sampling rate.
    N : number of samples
    KEYS : dictionary mapping each digit.

    Returns
    -------
    digit : detected digit.
```

```
    '''
    Freqs = np.array([[697.0, 770.0, 852.0, 941.0],[1209.0, 1336.0, 1477.0,
0]])
    res = float(Fs)/N # frequency resolution
    peak_idx = np.argsort(-x)[0:2] # indices of peak values in FFT
    idx = []

    # loop for finding the nearest DTMF frequencies in a tone
    for i in range(len(Freqs)):
        dist1 = abs(abs(Fs - Freqs[i]) - (peak_idx[0] * res))
        a =  np.argsort(dist1)[0]
        dist2 = abs(abs(Fs - Freqs[i]) - (peak_idx[1] * res))
        b =  np.argsort(dist2)[0]
        if dist1[a] < dist2[b]:
            idx.append(a)
        else:
            idx.append(b)

    digit = KEYS[idx[0]][idx[1]]
    return digit

data = np.loadtxt('touchtones.dat')
data = data[:,1]
Fs = 1000

plt.figure(1)
plt.plot(data)
plt.xlabel('Number of samples')
plt.ylabel('Sample Value')
plt.title('Original Signal in Time domain')

# dictionary mapping each digit
KEYS = [['1', '2', '3'], ['4', '5', '6'], ['7', '8', '9'], ['*', '0', '#']]
# normalize signal between +1 and -1
norm_data = (2*((data-np.min(data))/(np.max(data)-np.min(data))))-1

# loop for removing noise
for i in range(len(norm_data)):
    if (norm_data[i] >= 0.04) and (norm_data[i] <= 0.085):
        norm_data[i] = 0

plt.figure(2)
plt.plot(norm_data)
plt.xlabel('Number of samples')
plt.ylabel('Sample Value')
plt.title('Normalised Signal in Time domain')

edges = find_edges(norm_data) # find edges of each individual tone
digits = []

# loop for splitting data into individual tone segments and detecting
digits
for edge in edges:
    toneFT = np.abs(np.fft.fft(norm_data[edge[0]:edge[1]])) # compute FFT
of tone segment
    N = len(toneFT)
    toneFT_mirror = toneFT[0:int(N/2)] / N
    digits.append(decode_digit(toneFT_mirror, Fs, N, KEYS))

print(digits)
plt.show()
```